

The Cambrian Explosion of Symmetric Techniques for Advanced Protocols

Léo Perrin

Inria, Paris
`leo.perrin@inria.fr`

2nd of September 2026



Cambrian Explosion?



In this talk

Is there a **revolution** going on in symmetric cryptography?

In this talk

Is there a **revolution** going on in symmetric cryptography?

absolutely not.

In this talk

Is there a **revolution** going on in symmetric cryptography?

absolutely not.

Design criteria are changing, **again.**

In this talk

Is there a **revolution** going on in symmetric cryptography?

Part 1.1-2

absolutely not.

Design criteria are changing, **again.**

Part 1.3

Part 1

- 1 What are symmetric primitives?
- 2 How do we build secure symmetric primitives?
- 3 On their design constraints

In this talk

Is there a **revolution** going on in symmetric cryptography?

Part 1.1-2

absolutely not.

Design criteria are changing, **again.**

Part 1.3

Part 2

Part 1

- 1 What are symmetric primitives?
- 2 How do we build secure symmetric primitives?
- 3 On their design constraints

Part 2

- 1 Securing computations vs. data
- 2 Arithmetization-Orientation?

In this talk

Is there a **revolution** going on in symmetric cryptography?

Part 1.1-2

absolutely not.

Conclusion

Design criteria are changing, **again.**

Part 1.3

Part 2

Part 1

- 1 What are symmetric primitives?
- 2 How do we build secure symmetric primitives?
- 3 On their design constraints

Part 2

- 1 Securing computations vs. data
- 2 Arithmetization-Orientation?

Outline

- 1 "Advanced" Protocols: the Reason Behind Some Changes
- 2 A Revolution?

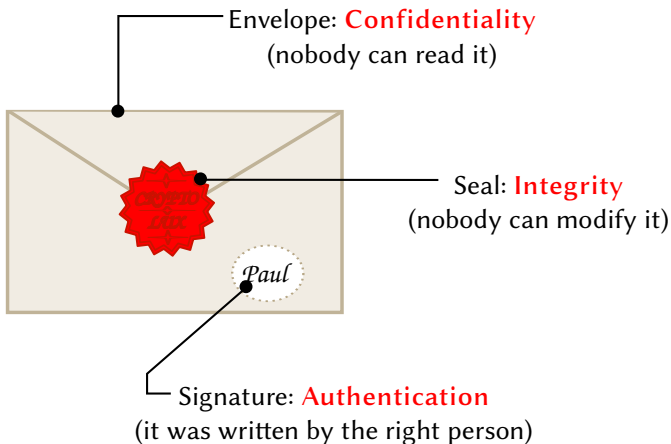
Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

2 A Revolution?

Securing Data

Usually, we secure **data** (at rest or in transit).



Securing Computation

More and more protocols intend to secure **computations**.

FHE Fully **H**omomorphic **E**ncryption

MPC Multi **P**arty **C**omputations

ZK-* Zero **K**nowledge- [proof, argument...]

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

■ (Fully) Homomorphic Encryption

■ Multi-Party Computations

■ Zero-Knowledge

■ One Approach to Rule Them All (?): Arithmetization

2 A Revolution?

FHE

Goal

Allow a third party to perform some operations on encrypted ciphertext that correspond to meaningful operations on the corresponding plaintext.

FHE

Goal

Allow a third party to perform some operations on encrypted ciphertext that correspond to meaningful operations on the corresponding plaintext.

A form of commutation

FHE

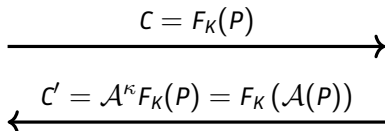
Goal

Allow a third party to perform some operations on encrypted ciphertext that correspond to meaningful operations on the corresponding plaintext.

A form of commutation

Alice

Bob



F_K is a homomorphic cipher,
not a block cipher!

FHE

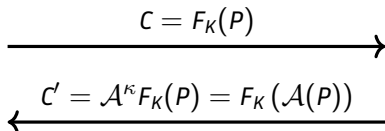
Goal

Allow a third party to perform some operations on encrypted ciphertext that correspond to meaningful operations on the corresponding plaintext.

A form of commutation

Alice

Bob



F_K is a homomorphic cipher,
not a block cipher!

An example of (not F)HE

XOR-ing a constant to a ciphertext obtained using a stream cipher XORs the same constant in the plaintext:

$$C \oplus t = (P \oplus K) \oplus t = (P \oplus t) \oplus K$$

The Symmetric Crypto They Need: Transciphering

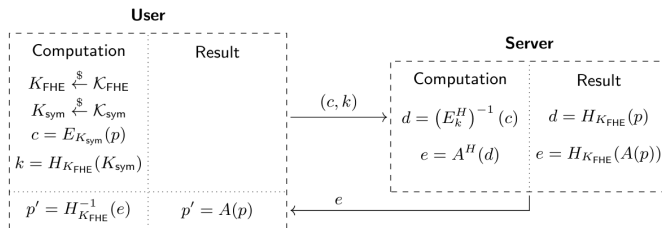


Fig. 1: The principle of transciphering, where E is a symmetric cipher (with secret key K_{sym} sampled from the space $\mathcal{K}_{\text{sym}}$), H is a fully homomorphic cipher (with private key K_{FHE} sampled from the space $\mathcal{K}_{\text{FHE}}$), E^H is a homomorphic evaluation of E , A corresponds to some arbitrary operations, and A^H to their homomorphic evaluation.

source: *Transistor: a TFHE-friendly Stream Cipher*

<https://eprint.iacr.org/2025/282>

The Symmetric Crypto They Need: Transciphering

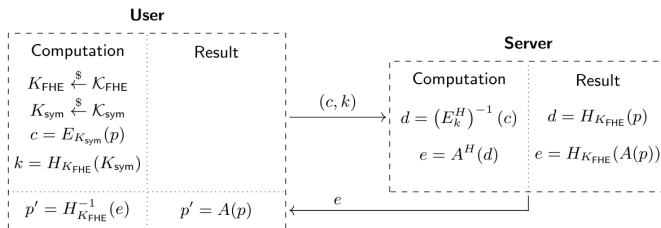


Fig. 1: The principle of transciphering, where E is a symmetric cipher (with secret key K_{sym} sampled from the space $\mathcal{K}_{\text{sym}}$), H is a fully homomorphic cipher (with private key K_{FHE} sampled from the space $\mathcal{K}_{\text{FHE}}$), E^H is a homomorphic evaluation of E , A corresponds to some arbitrary operations, and A^H to their homomorphic evaluation.

A symmetric encryption
algorithm with a
high throughput when
evaluated
homomorphically

source: *Transistor: a TFHE-friendly Stream Cipher*

<https://eprint.iacr.org/2025/282>

The case of TFHE

Operates on $\mathbb{Z}/m\mathbb{Z}$, where m can be anything, though: more efficient if m is smaller.

The case of TFHE

Operates on $\mathbb{Z}/m\mathbb{Z}$, where m can be anything, though: more efficient if m is smaller.

Operations allowed

Linear Combinations $\sum_i \alpha_i x_i$, where the α_i are constant while x_i is input/key dependent.

- Costs almost nothing in terms of time/communication complexity...
- But **noise** increases

The case of TFHE

Operates on $\mathbb{Z}/m\mathbb{Z}$, where m can be anything, though: more efficient if m is smaller.

Operations allowed

Linear Combinations $\sum_i \alpha_i x_i$, where the α_i are constant while x_i is input/key dependent.

- Costs almost nothing in terms of time/communication complexity...
- But **noise** increases

PBS (Programmable BootStrap) $y \leftarrow S(x)$

- Very time consuming...
- .. But resets the noise to a **base level**

The case of TFHE

Operates on $\mathbb{Z}/m\mathbb{Z}$, where m can be anything, though: more efficient if m is smaller.

Operations allowed

Linear Combinations $\sum_i \alpha_i x_i$, where the α_i are constant while x_i is input/key dependent.

- Costs almost nothing in terms of time/communication complexity...
- But **noise** increases

PBS (Programmable BootStrap) $y \leftarrow S(x)$

- Very time consuming...
- .. But resets the noise to a **base level**
- Can be composed with **arbitrary table lookups!**

The case of TFHE

Operates on $\mathbb{Z}/m\mathbb{Z}$, where m can be anything, though: more efficient if m is smaller.

Operations allowed

Linear Combinations $\sum_i \alpha_i x_i$, where the α_i are constant while x_i is input/key dependent.

- Costs almost nothing in terms of time/communication complexity...
- But **noise** increases

PBS (Programmable BootStrap) $y \leftarrow S(x)$

- Very time consuming...
- .. But resets the noise to a **base level**
- Can be composed with **arbitrary table lookups!**

(not T)FHE operates differently, but **noise** is still present

Examples of stream ciphers for transciphering

Elisabeth-4 [?]

$q = 2^4$ Can be linearized [?]

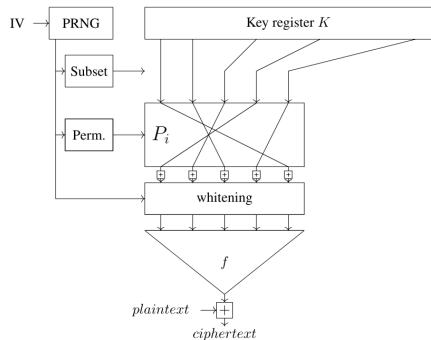


Fig. 1: The group filter permutator design

Examples of stream ciphers for transciphering

Elisabeth-4 [?]

$q = 2^4$ Can be linearized [?]

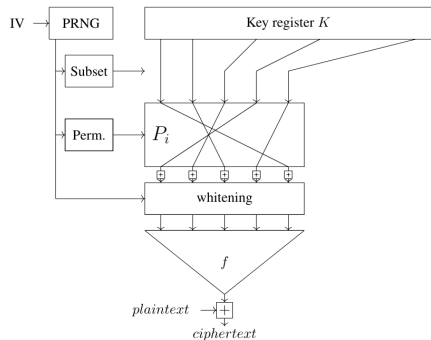


Fig. 1: The group filter permutator design

· ASTA

$q = 2$ or large prime

Many, *many* variants (Rasta, Dasta, Pasta, Masta)

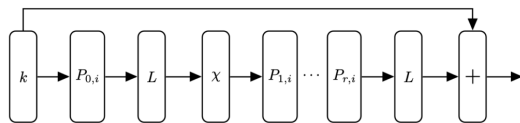


Figure 2: Generation of i -th block of DASTA.

source: *Dasta – Alternative Linear Layer for Rasta*

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- Zero-Knowledge
- One Approach to Rule Them All (?): Arithmetization

2 A Revolution?

Multi-Party Computations

Goal

Allow multiple parties to evaluate a function together even if some parties are not trustworthy.

Multi-Party Computations

Goal

Allow multiple parties to evaluate a function together even if some parties are not trustworthy.

Example

Shamir's secret sharing: $n + 1$ points are necessary to interpolate a degree n function.

Multi-Party Computations

Goal

Allow multiple parties to evaluate a function together even if some parties are not trustworthy.

Example

Shamir's secret sharing: $n + 1$ points are necessary to interpolate a degree n function.

Applications

- Masking (the side-channel attack counter-measure)
- MPC-in-the-head paradigm (e.g. for Picnic signatures)
- Trojan resilience
- ...

The SymCry They Need: a Lot of Different Things

Masking-friendly (Tweakable) BCs. Low number of multiplications in the underlying field.
Examples: `small-pSquare [?]`, `Fantomas [?]`,...

The SymCry They Need: a Lot of Different Things

Masking-friendly (Tweakable) BCs. Low number of multiplications in the underlying field.

Examples: `small-pSquare` [?], `Fantomas` [?],...

Picnic-friendly BCs. Low number of multiplications, security only for a single query.

Example: `LowMC` [?]

The SymCry They Need: a Lot of Different Things

Masking-friendly (Tweakable) BCs. Low number of multiplications in the underlying field.

Examples: `small-pSquare` [?], `Fantomas` [?],...

Picnic-friendly BCs. Low number of multiplications, security only for a single query.

Example: `LowMC` [?]

Trojan Resilient BCs Reliance only on linear operations (but over different ring/fields).

The SymCry They Need: a Lot of Different Things

Masking-friendly (Tweakable) BCs. Low number of multiplications in the underlying field.

Examples: small-pSquare [?], Fantomas [?],...

Picnic-friendly BCs. Low number of multiplications, security only for a single query.

Example: LowMC [?]

Trojan Resilient BCs Reliance only on linear operations (but over different ring/fields).

Example: MOE [?]

Pseudo-Correlated Functions

A new challenger!

At the start of some MPC protocols, it is necessary to share some bits that are correlated between the participants.

Very low multiplicative depth ·

Pseudo-Correlated Functions

A new challenger!

At the start of some MPC protocols, it is necessary to share some bits that are correlated between the participants.

Very low multiplicative depth · Very low data complexity ·

Pseudo-Correlated Functions

A new challenger!

At the start of some MPC protocols, it is necessary to share some bits that are correlated between the participants.

Very low multiplicative depth · Very low data complexity · **Only known plaintext, no chosen plaintext!**

Pseudo-Correlated Functions

A new challenger!

At the start of some MPC protocols, it is necessary to share some bits that are correlated between the participants.

Very low multiplicative depth · Very low data complexity · **Only known plaintext, no chosen plaintext!**

Crypto DarkMatter [?]

$$F_k(x) := \left(\sum_{i=0}^{n-1} k_i x_i \mod 2 + \sum_{i=0}^{n-1} k_i x_i \mod 3 \right) \mod 2, \quad \text{for } x \in \{0, 1\}^n.$$

Pseudo-Correlated Functions

A new challenger!

At the start of some MPC protocols, it is necessary to share some bits that are correlated between the participants.

Very low multiplicative depth · Very low data complexity · **Only known plaintext, no chosen plaintext!**

Crypto DarkMatter [?]

$$F_k(x) := \left(\sum_{i=0}^{n-1} k_i x_i \mod 2 + \sum_{i=0}^{n-1} k_i x_i \mod 3 \right) \mod 2, \quad \text{for } x \in \{0, 1\}^n.$$

VDLPN [?]

$$f_k(x) = \bigoplus_{i=1}^D \bigoplus_{j=1}^w \bigwedge_{\ell=1}^i (x_{i,j,\ell} \oplus k_{i,j,\ell}),$$

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- **Zero-Knowledge**
- One Approach to Rule Them All (?): Arithmetization

2 A Revolution?

Zero-Knowledge

Principle

I want to convince you I know a secret **without revealing it**

Zero-Knowledge

Principle

I want to convince you I know a secret **without revealing it**

A generic goal

To be able to prove/argue that a function was evaluated correctly without revealing its input.

Zero-Knowledge

Principle

I want to convince you I know a secret **without revealing it**

A generic goal

To be able to prove/argue that a function was evaluated correctly without revealing its input.
Ex: Sudoku time!

Zero-Knowledge

Principle

I want to convince you I know a secret **without revealing it**

A generic goal

To be able to prove/argue that a function was evaluated correctly without revealing its input.
Ex: Sudoku time!

Applications

- Offload computation to untrusted 3rd parties
- Electronic vote (mixnets)
- Signatures (e.g. with Solid)

Zero-Knowledge

Principle

I want to convince you I know a secret **without revealing it**

A generic goal

To be able to prove/argue that a function was evaluated correctly without revealing its input.
Ex: Sudoku time!

Applications

- Offload computation to untrusted 3rd parties
- Electronic vote (mixnets)
- Signatures (e.g. with Solid)
- BLOCKCHAIN!!!

The Symetric Crypto They Need: Easily Verifiable Hash Functions

(what follows is a simplification)

They want **hash functions** where the round function has a...

Low Degree Verification

The Symetric Crypto They Need: Easily Verifiable Hash Functions

(what follows is a simplification)

They want **hash functions** where the round function has a...

Low Degree Verification

A function $F : \mathbb{F}_q \rightarrow \mathbb{F}_q$ admits a *low degree verification* if there exists a low degree function $G : \mathbb{F}_q^2 \rightarrow \mathbb{F}_q$ such that

$$F(x) - z = 0 \Leftrightarrow G(x, z) = 0.$$

The Symetric Crypto They Need: Easily Verifiable Hash Functions

(what follows is a simplification)

They want **hash functions** where the round function has a...

Low Degree Verification

A function $F : \mathbb{F}_q \rightarrow \mathbb{F}_q$ admits a *low degree verification* if there exists a low degree function $G : \mathbb{F}_q^2 \rightarrow \mathbb{F}_q$ such that

$$F(x) - z = 0 \Leftrightarrow G(x, z) = 0.$$

Direct evaluation if F has a low degree, we are fine: $G(x, z) = F(x) - z$

The Symetric Crypto They Need: Easily Verifiable Hash Functions

(what follows is a simplification)

They want **hash functions** where the round function has a...

Low Degree Verification

A function $F : \mathbb{F}_q \rightarrow \mathbb{F}_q$ admits a *low degree verification* if there exists a low degree function $G : \mathbb{F}_q^2 \rightarrow \mathbb{F}_q$ such that

$$F(x) - z = 0 \Leftrightarrow G(x, z) = 0.$$

Direct evaluation if F has a low degree, we are fine: $G(x, z) = F(x) - z$

Indirect evaluation if F^{-1} has a low degree, then $G(x, z) = x - F^{-1}(z)$

The Symetric Crypto They Need: Easily Verifiable Hash Functions

(what follows is a simplification)

They want **hash functions** where the round function has a...

Low Degree Verification

A function $F : \mathbb{F}_q \rightarrow \mathbb{F}_q$ admits a *low degree verification* if there exists a low degree function $G : \mathbb{F}_q^2 \rightarrow \mathbb{F}_q$ such that

$$F(x) - z = 0 \Leftrightarrow G(x, z) = 0.$$

Direct evaluation if F has a low degree, we are fine: $G(x, z) = F(x) - z$

Indirect evaluation if F^{-1} has a low degree, then $G(x, z) = x - F^{-1}(z)$

Generalization it is sufficient that F is **CCZ-equivalent** to a low degree function.

Example 1/4: MiMC

MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity

Martin Albrecht¹, Lorenzo Grassi³, Christian Rechberger^{2,3}, Arnab Roy², and Tyge Tiessen²

¹ Royal Holloway, University of London, UK
martinalbrecht@googlemail.com

² DTU Compute, Technical University of Denmark, Denmark
{arroy,crec,tyti}@dtu.dk

³ IAIK, Graz University of Technology, Austria
{christian.rechberger,lorenzo.grassi}@iaik.tugraz.at

Published at ASIACRYPT'16;

<https://eprint.iacr.org/2016/492.pdf>

■ Base field: $\mathbb{F}_q$, where e.g. $q = 2^{257}$

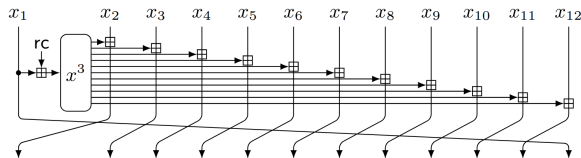
■ Round function:

$$R_i \begin{cases} \mathbb{F}_q & \rightarrow \mathbb{F}_q \\ x & \mapsto (x + c_i)^3 \end{cases}$$

where the *round constants* c_i have been generated randomly.

■ Number of rounds: $\ell \approx 90$

Example 2/4: gMiMC

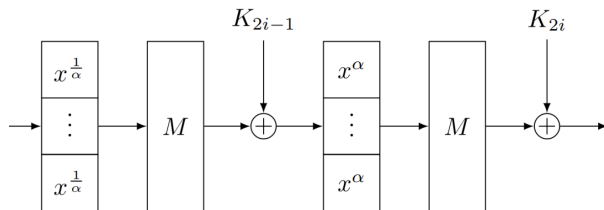


Published at ESORICS'19;
Albrecht, Perrin, Ramacher, Rechberger, Rotaru, Roy, Schofnegger

<https://eprint.iacr.org/2019/397.pdf>

- Base field: $\mathbb{F}_q$, where $q = 2^n$ or $q = p \geq 2^n, n \geq 64$
- Round function: see left
- Number of rounds: $\ell > 170$

Example 3/4: Rescue



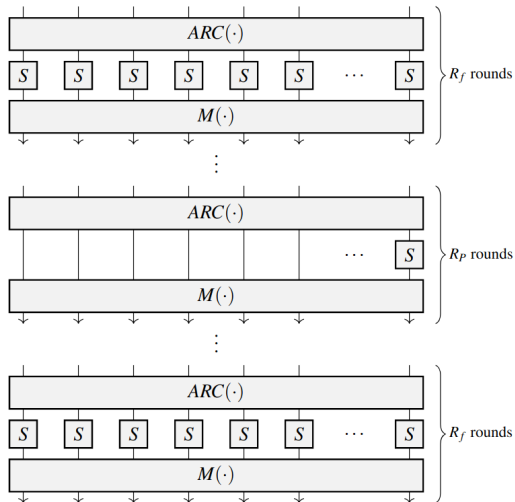
Published at ToSC'20(3);
Aly, Ashur, Ben-Sasson, Dhooghe, Szepieniec

<https://tosc.iacr.org/index.php/ToSC/article/view/8695/8287>

- Base field: $\mathbb{F}_q$, where
 $q = p \geq 2^n, n \geq 64$
- Round function: see left; $\alpha = 3$
and M is a linear permutation of
 $\mathbb{F}_q^t$.
- Number of rounds: $\ell \approx 10$

Verification: $P_i(x_i) == Q_i(x_{i+1})$, where P_i is a half round,
and Q_i is the **inverse** of the other half!

Example 4/4: Poseidon



Published at USENIX'21;
Grassi, Khovratovich, Rechberger, Roy, Schofnegger
<https://eprint.iacr.org/2019/458.pdf>

- Base field: $\mathbb{F}_q$, where $q = p \geq 2^n$, $n \geq 64$
- Round function: $S(x) = x^3$, ARC add a round constant, and M is a linear permutation of $\mathbb{F}_q^t$.
- Number of rounds: $\ell = R_f + R_p \approx 50$

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- Zero-Knowledge
- One Approach to Rule Them All (?): Arithmetization

2 A Revolution?

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

This verification costs 4 R1CS constraints

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

This verification costs 4 R1CS constraints

- How to turn a computation into an arithmetic circuit depends on the operations allowed
- Its cost is also arithmetization-dependent—though low degree is usually welcome!

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

This verification costs **4 R1CS constraints**

- How to turn a computation into an arithmetic circuit depends on the **operations allowed**
- Its cost is also arithmetization-dependent—**though low degree is usually welcome!**
- Arithmetization over a field of odd size → **nonbinary ciphers**

A not basic at all example of arithmetization

The cost of each operation depends on the arithmetization!
Plonk $\neq$ R1CS

A not basic at all example of arithmetization

The cost of each operation depends on the arithmetization!
Plonk $\neq$ R1CS

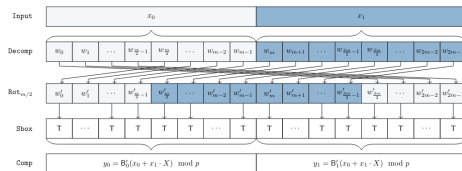


Figure 3: The Bar layer $B' : \mathbb{F}_{p^n} \rightarrow \mathbb{F}_{p^n}$ for $n = 2$ in detail, including the decomposition, the rotation, the S-box, and the composition.

source: *Skyscraper: Fast Hashing on Big Primes*,
<https://eprint.iacr.org/2025/058.pdf>

Symmetric Techniques for Advanced Protocols

MPC

FHE

ZK

Symmetric Techniques for Advanced Protocols

MPC

FHE

ZK

Masking

BGV

R1CS

MPC-in-the-head
(signatures...)

FV

AIR
...

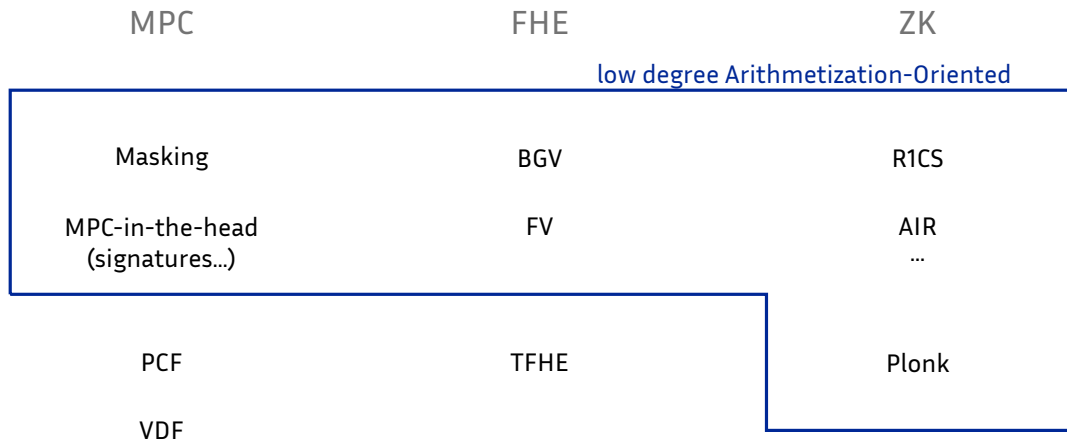
PCF

TFHE

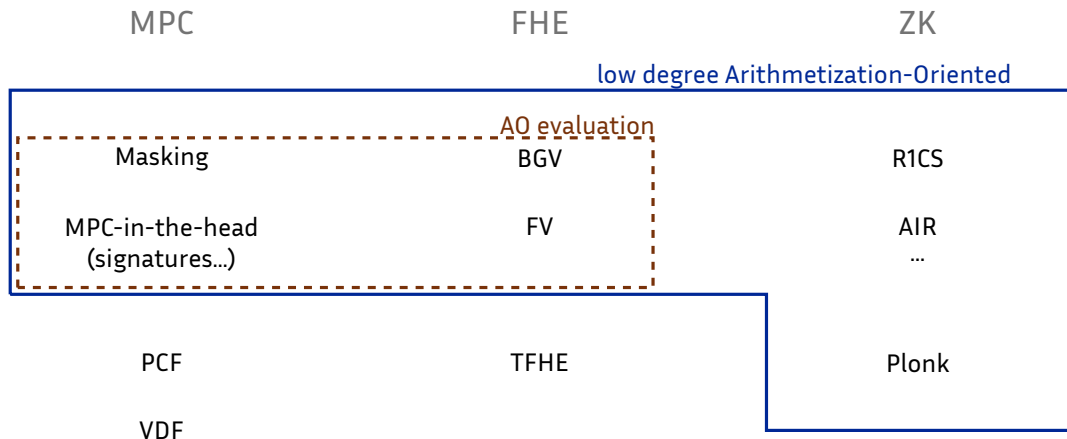
Plonk

VDF

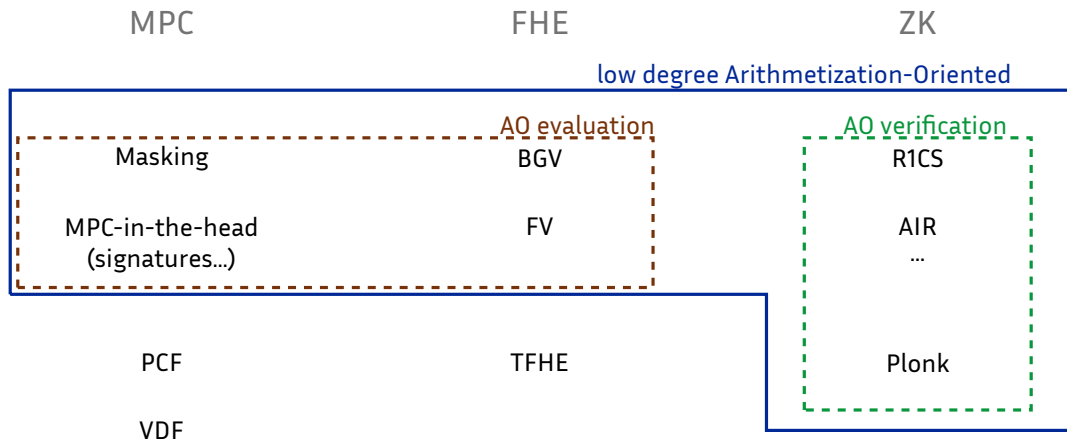
Symmetric Techniques for Advanced Protocols



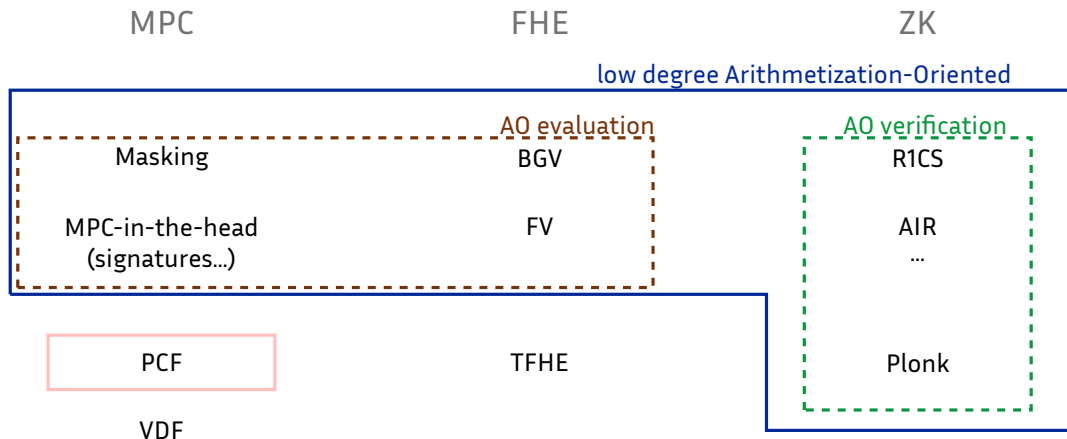
Symmetric Techniques for Advanced Protocols



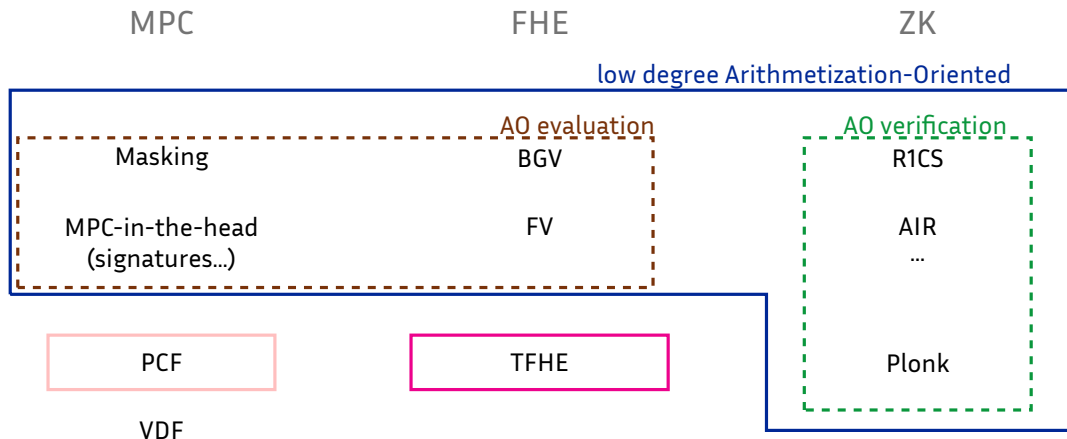
Symmetric Techniques for Advanced Protocols



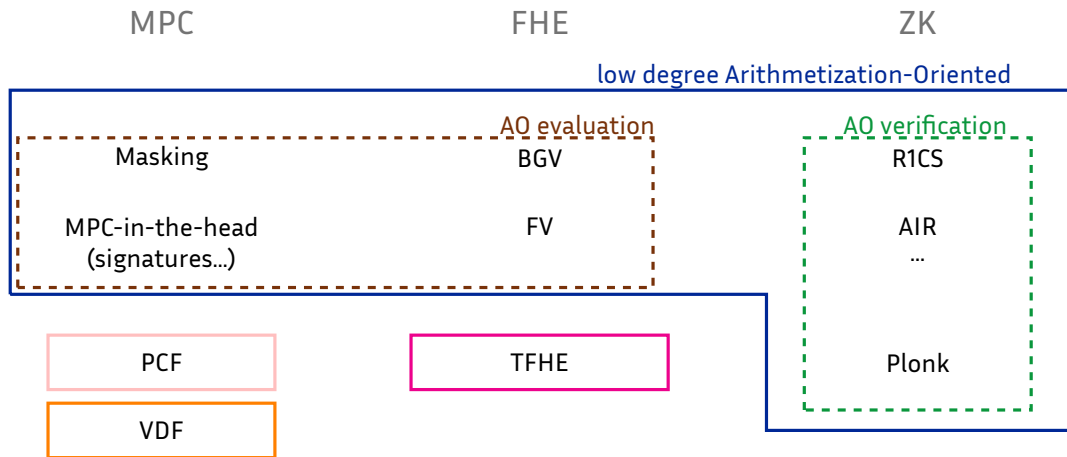
Symmetric Techniques for Advanced Protocols



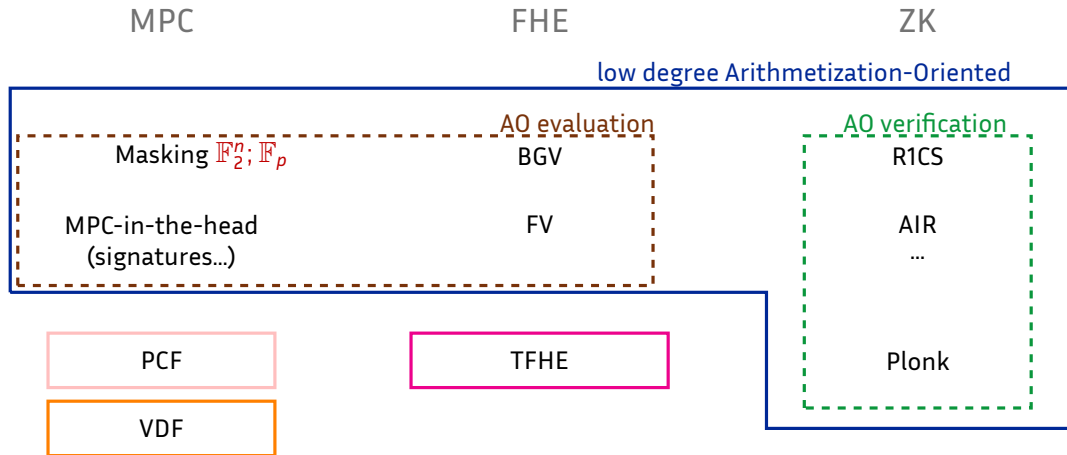
Symmetric Techniques for Advanced Protocols



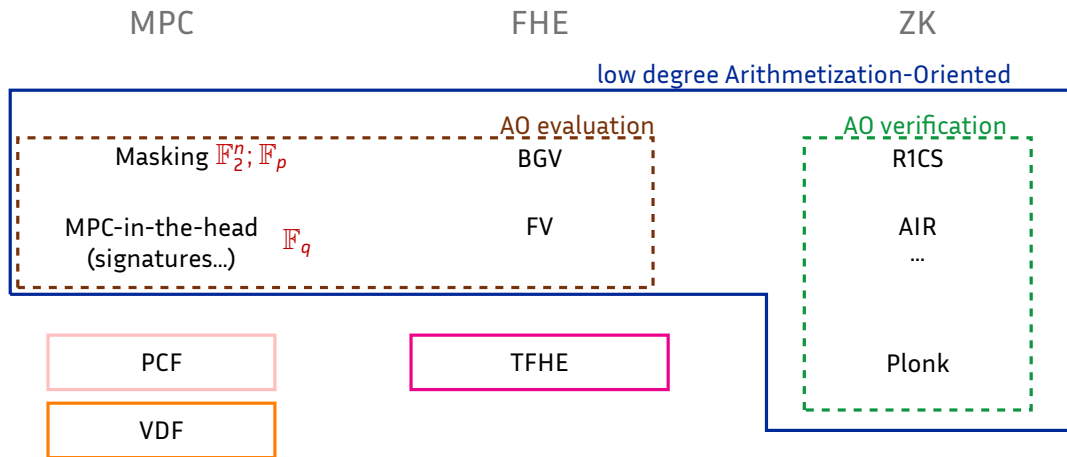
Symmetric Techniques for Advanced Protocols



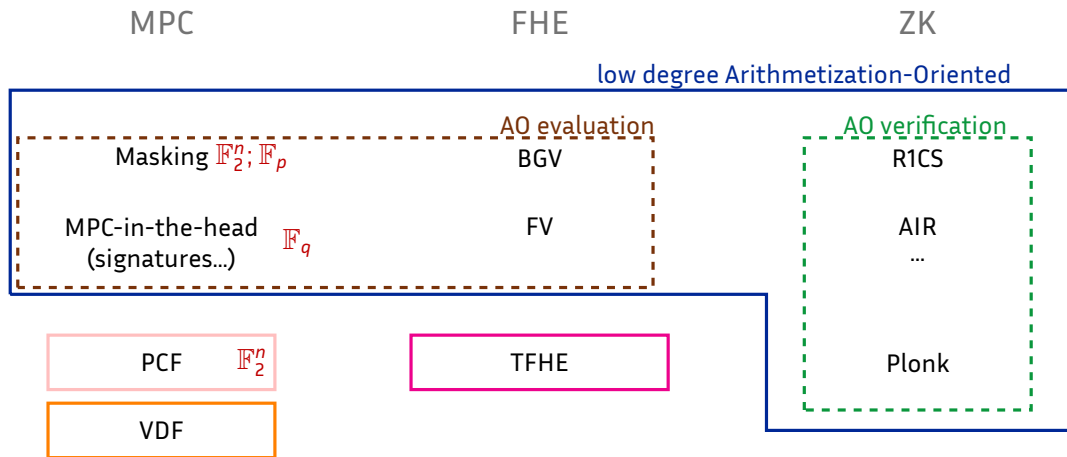
Symmetric Techniques for Advanced Protocols



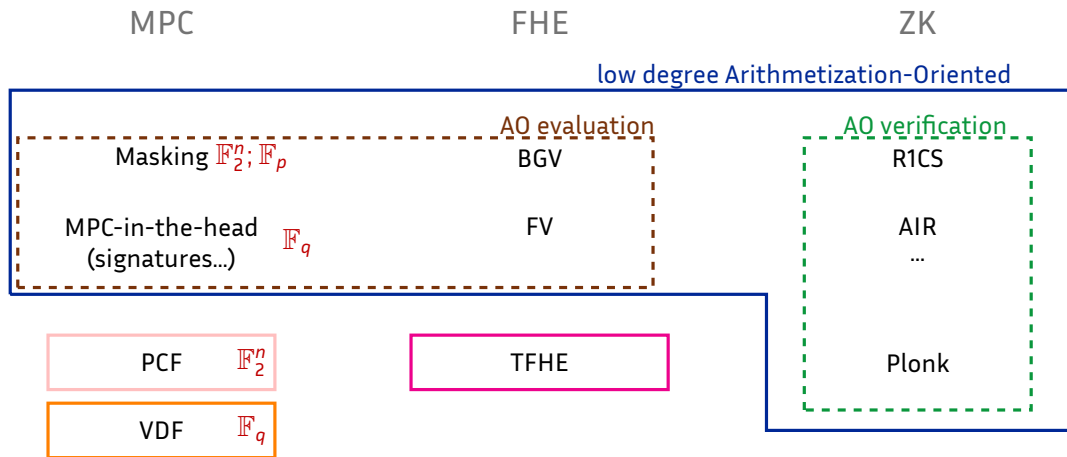
Symmetric Techniques for Advanced Protocols



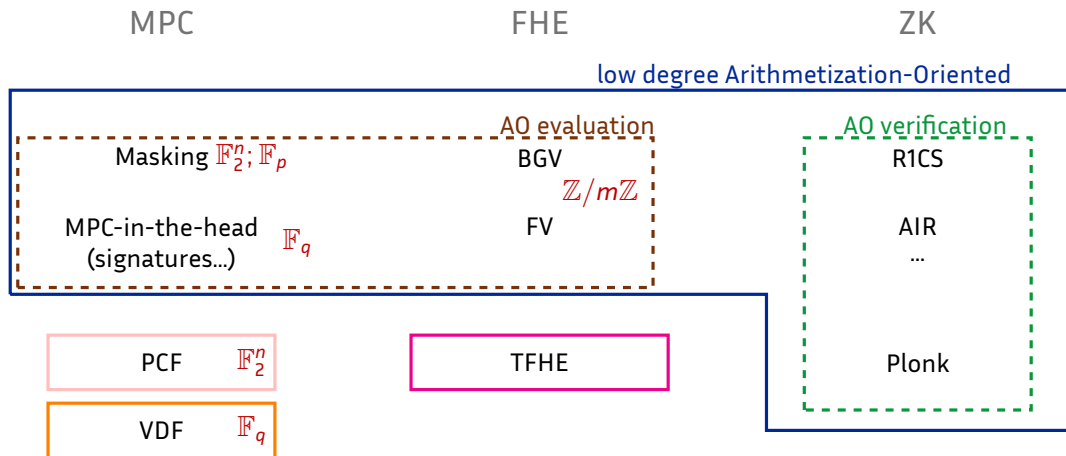
Symmetric Techniques for Advanced Protocols



Symmetric Techniques for Advanced Protocols



Symmetric Techniques for Advanced Protocols



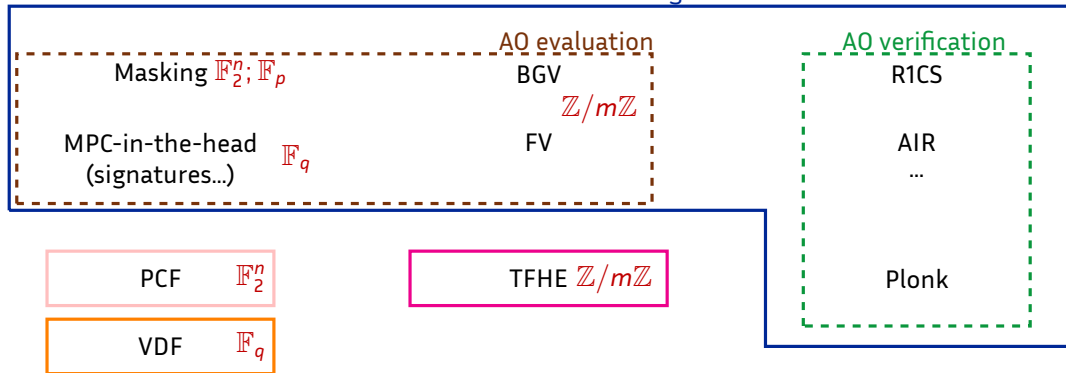
Symmetric Techniques for Advanced Protocols

MPC

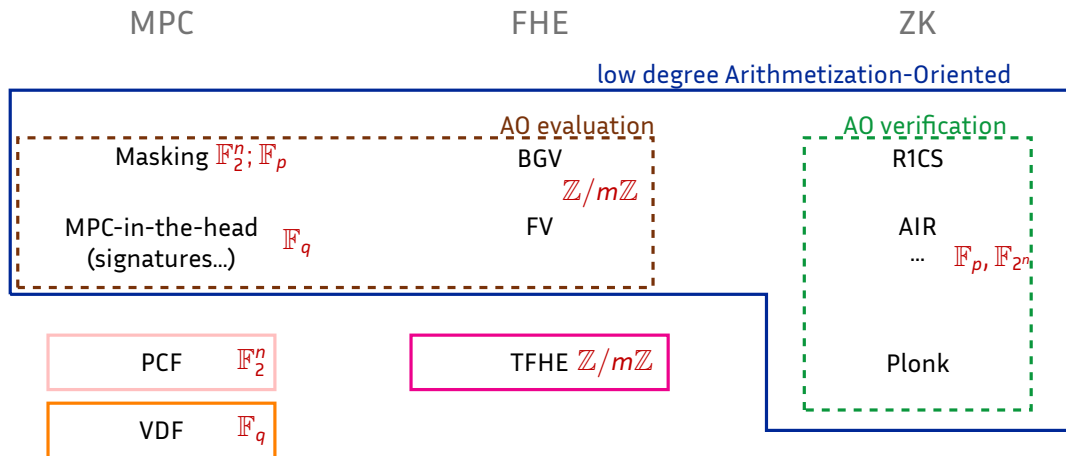
FHE

ZK

low degree Arithmetization-Oriented

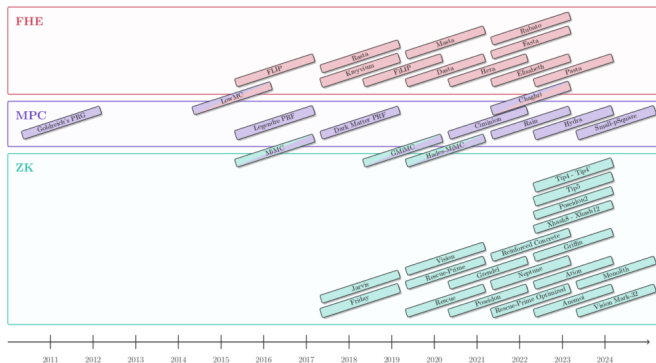


Symmetric Techniques for Advanced Protocols



A Cambrian Explosion

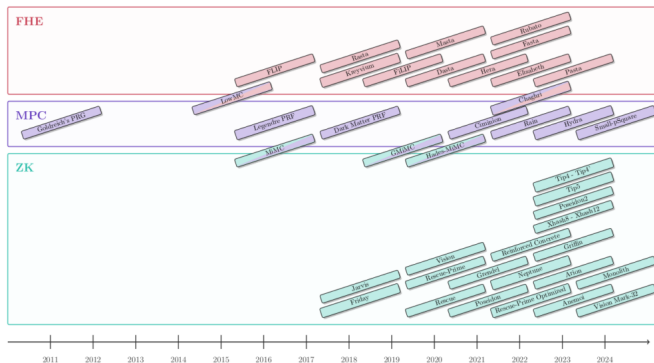
credit: Cl  mence Bouvier [?]



<https://stap-zoo.com/>

A Cambrian Explosion

credit: Cl  mence Bouvier [?]



<https://stap-zoo.com/>

An increased diversity of
design criteria leads to a
Cambrian explosion of new
symmetric primitives!

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

2 A Revolution?

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

2 A Revolution?

- What Is and Isn't Specific to STAPs

- Conclusion

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**
- **New tricks** for the initial security analysis

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**
- **New tricks** for the initial security analysis
- Tight **collaboration with implementers** $\neq$ the people doing the cryptanalysis

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**
- **New tricks** for the initial security analysis
- Tight **collaboration with implementers** $\neq$ the people doing the cryptanalysis
- Multiple **iterations** during the design to maximize efficiency

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**
- **New tricks** for the initial security analysis
- Tight **collaboration with implementers** $\neq$ the people doing the cryptanalysis
- Multiple **iterations** during the design to maximize efficiency

Dear audience, which specific primitive am I talking about?

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**
- **New tricks** for the initial security analysis
- Tight **collaboration with implementers** $\neq$ the people doing the cryptanalysis
- Multiple **iterations** during the design to maximize efficiency

Dear audience, which specific primitive am I talking about?
all of them!

Opinion 1

STAPs are nothing special: we (symmetric people) need to do what we always did.

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed.

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.
- Protocol flexibility means STAPs are often in fact **primitives generators**, i.e., algorithms **generating** primitives.

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.
- Protocol flexibility means STAPs are often in fact **primitives generators**, i.e., algorithms **generating** primitives.
- Low degree arithmetization implies low degree algebraic modeling
beware of algebraic attacks!

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.
- Protocol flexibility means STAPs are often in fact **primitives generators**, i.e., algorithms **generating** primitives.
- Low degree arithmetization implies low degree algebraic modeling

beware of algebraic attacks!

(there already was a mass extinction event, the **Freelunch attack**
(I'm pretty sure **Morten** will tell you more!))

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.
- Protocol flexibility means STAPs are often in fact **primitives generators**, i.e., algorithms **generating** primitives.
- Low degree arithmetization implies low degree algebraic modeling
beware of algebraic attacks!
(there already was a mass extinction event, the **Freelunch attack**
(I'm pretty sure **Morten** will tell you more!))

Opinion 2

Working over $\mathbb{F}_q$ (especially if low degree arithmetizations are needed) and the need for primitive generators introduce **new cryptanalysis vectors**

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.
- Protocol flexibility means STAPs are often in fact **primitives generators**, i.e., algorithms **generating** primitives.
- Low degree arithmetization implies low degree algebraic modeling
beware of algebraic attacks!
(there already was a mass extinction event, the **Freelunch attack**
(I'm pretty sure **Morten** will tell you more!))

Opinion 2

Working over $\mathbb{F}_q$ (especially if low degree arithmetizations are needed) and the need for primitive generators introduce **new cryptanalysis vectors**, **but design approaches will rely on tried and true methods**.

Primitive Overdose



credit: Diego Delso, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=108259695>

Cryptanalysis has not followed the design
explosion

Primitive Overdose



credit: Diego Delso, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=108259695>

Cryptanalysis has not followed the design
explosion

Potential Explanations

- Not all primitives are designed to **ease analysis**
- So many (too many) minor variants...

Primitive Overdose



credit: Diego Delso, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=108259695>

Cryptanalysis has not followed the design
explosion

Potential Explanations

- Not all primitives are designed to **ease analysis**
- So many (too many) minor variants...

Opinion 3

- 1 We need more **cryptanalysis!**
- 2 We must become better at handling **primitive generators.**

Plan of this Section

1 "Advanced" Protocols: the Reason Behind Some Changes

2 A Revolution?

- What Is and Isn't Specific to STAPs
- Conclusion

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**
- The **constraints** they impose **vary**, and **we should be careful about lumping them together**

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**
- The **constraints** they impose **vary**, and **we should be careful about lumping them together**
- A lot of new sets of constraints $\implies$ a lot of new designs (**Cambrian explosion**)...

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**
- The **constraints** they impose **vary**, and **we should be careful about lumping them together**
- A lot of new sets of constraints $\implies$ a lot of new designs (**Cambrian explosion**)...
- ... but not enough cryptanalysis.

We need more cryptanalysis!

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**
- The **constraints** they impose **vary**, and **we should be careful about lumping them together**
- A lot of new sets of constraints $\implies$ a lot of new designs (**Cambrian explosion**)...
- ... but not enough cryptanalysis.

We need more cryptanalysis!

Design criteria dictate the work of symmetric cryptographers
they always have, and always will!

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**
- The **constraints** they impose **vary**, and **we should be careful about lumping them together**
- A lot of new sets of constraints $\implies$ a lot of new designs (**Cambrian explosion**)...
- ... but not enough cryptanalysis.

We need more cryptanalysis!

Design criteria dictate the work of symmetric cryptographers
they always have, and always will!

Thank you!